

[Capacity AR](#)[api-curls.md](#)

Curls de prueba de la API

Guía de prueba de la API deployada para que los compañeros del equipo puedan probar los endpoints sin tener que levantar nada local.

Links principales

- **API base:** <https://capacity-ar-ap.onrender.com>
- **Swagger UI (probar desde el navegador):** <https://capacity-ar-ap.onrender.com/docs>
- **Health check:** <https://capacity-ar-ap.onrender.com/api/health>

Antes de empezar

El plan free de Render **duerme el servidor tras 15 minutos sin tráfico**. El primer request después de un rato tarda entre 30 y 50 segundos en despertar el contenedor. Los siguientes vuelan.

Para "calentar" el server antes de una prueba o demo:

```
curl https://capacity-ar-ap.onrender.com/api/health
```

Cuando devuelva `{"status":"ok","environment":"production"}` ya está listo.

Endpoints disponibles

Users

Método	Ruta	Descripción
GET	/api/users	Listar usuarios (con paginación)
POST	/api/users	Crear usuario
GET	/api/users/{id}	Obtener un usuario por id
PATCH	/api/users/{id}	Actualizar parcialmente
DELETE	/api/users/{id}	Borrar usuario

Roles válidos: [student](#), [tutor](#), [company_admin](#), [admin](#).

Gap analyses (carga de documentos → summary)

Método	Ruta	Descripción
POST	/api/gap-analyses	Subir 2 documentos (estudiante + oferta), Groq extrae GapReport, guarda en BD. NO dispara plan.
GET	/api/gap-analyses/{id}	Ver un gap por id
GET	/api/students/{email}/gap-analyses	Listar gaps de un estudiante
POST	/api/students/{email}/generate-learning-path	A partir del último gap pendiente del estudiante, dispara la generación del learning_path

Learning paths

Método	Ruta	Descripción
POST	/api/learning-paths	Crear plan a partir de un GapReport JSON (sin documentos)
GET	/api/learning-paths	Listar planes
GET	/api/learning-paths/{id}	Obtener un plan por id
GET	/api/students/{email}/learning-paths	Listar planes de un estudiante

Attempts (ejercicios resueltos por el alumno)

Método	Ruta	Descripción
POST	/api/attempts	Registrar intento, evalúa y devuelve feedback con IA
GET	/api/attempts/{id}	Ver un attempt
GET	/api/students/{email}/attempts	Historial de un estudiante

USERS — curls

Crear

```
curl -X POST https://capacity-ar-ap.onrender.com/api/users \
  -H "Content-Type: application/json" \
  -d
'{"first_name":"Ana","last_name":"Perez","email":"ana@example.com","role":"student"}
'
```

Roles válidos: `student`, `tutor`, `company_admin`, `admin`. Reintento con el mismo email devuelve `409`.

Listar / ver / actualizar / borrar

```
curl https://capacity-ar-ap.onrender.com/api/users
curl "https://capacity-ar-ap.onrender.com/api/users?offset=0&limit=10"
curl https://capacity-ar-ap.onrender.com/api/users/1
curl -X PATCH https://capacity-ar-ap.onrender.com/api/users/1 \
  -H "Content-Type: application/json" -d '{"last_name":"Gonzalez"}'
curl -X DELETE https://capacity-ar-ap.onrender.com/api/users/2
```

GET `/api/users/9999` → `404`. DELETE exitoso → `204` sin body.

GAP ANALYSES — curls

Flujo en dos pasos:

1. **Subir documentos** del estudiante y de la oferta → Groq extrae un GapReport y lo guarda. Rápido (3-5s). NO dispara plan.
2. **Disparar el plan más tarde** identificando al estudiante por email. El sistema busca el último gap pendiente y llama a `learning_paths`. Lento (15-25s), pero aislado del primer paso.

Aceptamos `PDF`, `DOCX` y `TXT`. Máximo 5MB por archivo, mínimo 50 caracteres tras extraer texto.

Paso 1 — Subir documentos y generar summary

```
curl -X POST https://capacity-ar-ap.onrender.com/api/gap-analyses \
  -F "student_email=sofia.maldonado@example.com" \
  -F "company_email=data-talent@globant.com" \
  -F "student_doc=@/ruta/al/cv_sofia.pdf" \
  -F "position_doc=@/ruta/a/oferta_globant.pdf"
```

Respuesta `201` con `id`, `summary`, `readiness_score`, `gap_report.skills` calculadas, `learning_path_id: null`.

Tip rápido: podés probar con dos `.txt`. Acordate del `@` antes de la ruta (es la convención de curl para subir archivos).

Paso 2 — Disparar el plan a partir del email

Toma el último gap pendiente (`learning_path_id IS NULL`) del estudiante:

```
curl -X POST https://capacity-ar-ap.onrender.com/api/students/sofia.maldonado@example.com/generate-learning-path
```

Respuesta `201` con `{ gap_analysis, learning_path }`. El `gap_analysis` ahora tiene `learning_path_id` apuntando al plan recién creado.

Consultar / listar

```
# Un gap por id
curl https://capacity-ar-ap.onrender.com/api/gap-analyses/1

# Todos los gaps de un estudiante (devuelve array, ordenado del más reciente al más viejo)
curl https://capacity-ar-ap.onrender.com/api/students/sofia.maldonado@example.com/gap-analyses
```

Casos de error

- Archivo con extensión distinta a `.pdf/.docx/.txt` → `422`.
- Archivo > 5MB → `413`.
- Texto extraído < 50 caracteres (PDF escaneado sin OCR, archivo casi vacío) → `422`.
- Email inválido → `422`.
- `generate-learning-path` sin gap pendiente para ese email → `404`.
- Groq devuelve JSON inválido o falla → `502` con detalle.

Cómo funciona con varios summaries por email

El alumno puede subir documentos varias veces — cada call crea una fila nueva. Cuando llama a `generate-learning-path`, el sistema toma el **último gap con `learning_path_id IS NULL`** (LIFO). Si quiere generar planes para los anteriores también, vuelve a llamar al endpoint hasta que devuelva `404`.

Verificación en Neon

```
-- Resumen de gaps por estudiante
SELECT id, student_email, company_email, readiness_score,
       learning_path_id, created_at
FROM gap_analyses
ORDER BY id DESC LIMIT 10;

-- Ver el GapReport completo extraído por Groq
SELECT id, gap_report_json FROM gap_analyses WHERE id = 1;

-- Conectar gap con el plan generado
SELECT ga.id AS gap_id, ga.student_email, ga.summary,
       lp.id AS plan_id, lp.target_role_title
FROM gap_analyses ga
LEFT JOIN learning_paths lp ON lp.id = ga.learning_path_id
ORDER BY ga.id DESC;
```

LEARNING PATHS — curls

Crear plan desde un GapReport

```
curl -X POST https://capacity-ar-ap.onrender.com/api/learning-paths \
-H "Content-Type: application/json" \
-d '{
  "student": {
    "name": "Ana Perez",
    "email": "ana@example.com",
    "skills": [{ "name": "Git", "level": 1 }, { "name": "SQL", "level": 2 }],
    "interests": ["backend", "logistica"]
  },
  "company": { "name": "Empresa Logistica SA" },
  "target_role": {
    "title": "Backend Developer Junior",
    "required_skills": [
      { "name": "Git", "level": 3, "priority": "HIGH" },
      { "name": "SQL", "level": 3, "priority": "HIGH" },
      { "name": "HTTP APIs", "level": 2, "priority": "MEDIUM" }
    ]
  },
  "summary": "Ana necesita reforzar Git, SQL y HTTP APIs.",
  "readiness_score": 45,
  "skills": [
    { "name": "Git", "current_level": 1, "required_level": 3, "gap_level": 2,
      "priority": "HIGH", "status": "MISSING" },
    { "name": "SQL", "current_level": 2, "required_level": 3, "gap_level": 1,
      "priority": "HIGH", "status": "NEEDS_WORK" },
    { "name": "HTTP APIs", "current_level": 0, "required_level": 2, "gap_level": 2,
      "priority": "MEDIUM", "status": "MISSING" }
  ]
}'
```

Respuesta `201` con `id`, `status: "ACTIVE"`, `generator_used`, módulos por skill MISSING/NEEDS_WORK y cada módulo con units `pasion` / `play` / `practica`.

Listar / ver / por estudiante

```
curl https://capacity-ar-ap.onrender.com/api/learning-paths
curl https://capacity-ar-ap.onrender.com/api/learning-paths/1
curl https://capacity-ar-ap.onrender.com/api/students/ana@example.com/learning-paths
```

Casos de error

- Todas las skills en `READY` → `409 Conflict` (no genera plan vacío).
- `GET /api/learning-paths/9999` → `404`.
- Payload con campos faltantes → `422` con detalle del campo.

ATTEMPTS — curls

Identificación del ejercicio: 4 campos compuestos (`learning_path_id` + `module_index` + `unit_index` + `exercise_index`). El servidor abre el plan, navega esa posición y compara con el `expected_answer` original.

Responder un ejercicio

```
curl -X POST https://capacity-ar-ap.onrender.com/api/attempts \
-H "Content-Type: application/json" \
-d '{
  "student_email": "carlos.gamer@example.com",
  "learning_path_id": 7,
  "module_index": 0,
  "unit_index": 2,
  "exercise_index": 0,
  "answer": "B"
}'
```

Respuesta:

- `is_correct` + `score` (1.0 o 0.0).
- `ai_feedback` : mensaje determinístico si acertó, generado por IA en español rioplatense si falló (Paciencia — no revela la respuesta correcta, ofrece pista, invita a reintentar).
- `skill_mastery` : aciertos / total intentos en esa skill.
- `mastery_threshold_reached: true` cuando `skill_mastery >= 0.80` .

Consultar

```
curl https://capacity-ar-ap.onrender.com/api/attempts/1
curl https://capacity-ar-
ap.onrender.com/api/students/carlos.gamer@example.com/attempts
```

Errores esperados

- `learning_path_id` que no existe → `404` .
- Email del attempt distinto al del plan → `403 Forbidden` .

Mastery por skill (Neon)

```
SELECT skill_name,
       COUNT(*) AS attempts,
       SUM(CASE WHEN is_correct THEN 1 ELSE 0 END) AS aciertos,
       ROUND(SUM(CASE WHEN is_correct THEN 1.0 ELSE 0.0 END) / COUNT(*), 2) AS
mastery
FROM attempts
WHERE student_email = 'carlos.gamer@example.com'
GROUP BY skill_name;
```

RESOURCES — cómo verlos

Sin endpoints CRUD propios. Los recursos viajan dentro de cada `unit.resources[]` del plan generado.

```
-- Catálogo cacheado
SELECT skill_name, phase, type, title, source, url
FROM resources
WHERE is_active = TRUE
ORDER BY skill_name, phase, id;

-- Regenerar recursos de una skill
DELETE FROM resources WHERE skill_name = 'JavaScript';
```

Detalles en [resources-design.md](#).

Ver cómo la IA personaliza la respuesta

En cada respuesta del `POST /learning-paths` mirar:

- `generator_used` → "groq" o "gemini" según el proveedor activo. Si dice "mock" el LLM falló y cayó al fallback.
- `modules[0].units[0].content` → debe mencionar el nombre, los intereses y la empresa objetivo del estudiante.
- `modules[0].units[2].exercises` → 5 multiple_choice reales con A/B/C/D.
- `modules[0].units[*].resources` → 2-3 recursos externos por unit (videos, guías, sandboxes).

Para mostrar variabilidad: repetir el mismo body dos veces. Como `temperature=0.7`, el LLM produce contenidos distintos cada vez. Es la prueba clara de que no hay cache ni contenido hardcodeado.

Body de ejemplo (Frontend Junior con interés en videojuegos)

```
curl -X POST https://capacity-ar-ap.onrender.com/api/learning-paths \
-H "Content-Type: application/json" \
-d '{
  "student": {
    "name": "Carlos Mendez",
    "email": "carlos.gamer@example.com",
    "skills": [{ "name": "HTML", "level": 3 }],
    "interests": ["videojuegos", "esports"]
  },
  "company": { "name": "GameStudio Argentina" },
  "target_role": {
    "title": "Frontend Developer Junior",
    "required_skills": [{ "name": "JavaScript", "level": 3, "priority": "HIGH" }]
  },
  "summary": "Carlos necesita JavaScript.",
  "readiness_score": 30,
  "skills": [{ "name": "JavaScript", "current_level": 1, "required_level": 3,
```

```
"gap_level": 2, "priority": "HIGH", "status": "MISSING"}]
```

Para probar otro perfil, cambiar `student.interests`, `company.name`, `target_role.title` y las `skills` requeridas. El LLM va a adaptar las analogías y los ejercicios al nuevo contexto.

Tips

Ver status code + headers

```
curl -i https://capacity-ar-ap.onrender.com/api/users/9999
```

El `-i` muestra los headers incluyendo `HTTP/2 404`.

JSON con formato (requiere `jq`)

```
curl -s https://capacity-ar-ap.onrender.com/api/learning-paths | jq
```

Instalación: `sudo apt install jq` en Ubuntu/Debian, `brew install jq` en Mac.

Pegar todo desde Swagger en vez de curl

Entrar a <https://capacity-ar-ap.onrender.com/docs>, clicar el endpoint, "Try it out", editar el JSON y "Execute". Es lo más cómodo para una demo o para probar rápido sin terminal.

Contrato de datos

Contrato completo del GapReport y reglas de cálculo en [gap-engine-mvp.md](#).